

Documento de Arquitectura de Software (DAS)

Sistema de Gestión Cívica de Incidentes Viales ReportaTuCalle

Bryan Aldrin Chontay Campos

Paul Fran León Gonzales

Arturo Miguel Osorio Miranda

Erick Joel Sánchez Longa

Joaquín Marcelo Torres Reátegui

Proyecto ReportaTuCalle

July 12, 2026

Abstract—Este Documento de Arquitectura de Software (DAS) describe la arquitectura técnica del sistema ReportaTuCalle, una plataforma de gestión cívica que permite a ciudadanos reportar incidentes viales (baches, fugas de agua, semáforos dañados, acumulación de basura, entre otros) georreferenciados sobre un mapa interactivo, y a supervisores/administradores gestionar, priorizar y optimizar rutas de atención. Se documentan las vistas lógicas, de componentes, de despliegue y de procesos siguiendo el modelo 4+1 de Kruchten, el diseño de la base de datos relacional con soporte geoespacial (PostgreSQL + PostGIS), el modelo de clases del dominio, y se resaltan explícitamente los patrones de diseño Gang of Four (GoF) implementados a nivel de código para satisfacer los atributos de calidad de mantenibilidad, extensibilidad y bajo acoplamiento.

Index Terms—arquitectura hexagonal, arquitectura de software, patrones de diseño GoF, modelo de datos, PostGIS, Spring Boot, React, DAS

I. INTRODUCCIÓN

A. Propósito

El propósito de este documento es proporcionar una vista integral y multidimensional de la arquitectura del sistema ReportaTuCalle, sirviendo como referencia técnica para el equipo de desarrollo, revisores de calidad y nuevos integrantes del proyecto. Se detallan los componentes principales del sistema, sus interacciones, el modelo de datos persistente y las decisiones de diseño de software (patrones GoF) que sustentan la evolución del producto.

B. Alcance

El documento cubre: (i) la arquitectura de backend bajo el estilo Hexagonal (Puertos y Adaptadores) implementado en Java 21 con Spring Boot 3; (ii) la arquitectura de frontend tipo Single Page Application (SPA) con React 18; (iii) el diseño de la base de datos relacional PostgreSQL con la extensión espacial PostGIS; y (iv) los patrones de diseño de software aplicados manualmente en el código del dominio.

C. Definiciones, Siglas y Abreviaturas

- **DAS**: Documento de Arquitectura de Software.
- **GoF**: Gang of Four, autores del catálogo clásico de patrones de diseño orientado a objetos.
- **SSE**: Server-Sent Events, mecanismo de comunicación unidireccional servidor-cliente en tiempo real.
- **JWT**: JSON Web Token, estándar de token para autenticación stateless.

- **PostGIS**: extensión espacial de PostgreSQL para datos geográficos.
- **TSP**: Travelling Salesman Problem (problema del agente viajero).
- **MST**: Minimum Spanning Tree (árbol de expansión mínima).
- **DTO**: Data Transfer Object.
- **ORM**: Object-Relational Mapping.

II. METAS Y RESTRICCIONES ARQUITECTÓNICAS

A. Atributos de Calidad Priorizados

- **Mantenibilidad**: separación estricta entre dominio puro y detalles de infraestructura (JPA, Spring, HTTP) mediante Arquitectura Hexagonal.
- **Extensibilidad**: incorporación de nuevos algoritmos de optimización de rutas o nuevos canales de notificación sin modificar código existente (principio Abierto/Cerrado vía patrones Strategy y Factory Method).
- **Bajo acoplamiento / Alta cohesión**: organización del backend en módulos por contexto delimitado (bounded context): auth, user, report, category, media, notification y optimization, cada uno con su propio esquema de base de datos.
- **Disponibilidad de datos en tiempo real**: actualización inmediata de los tableros de administrador y ciudadano ante cambios de estado mediante Server-Sent Events (patrón Observer).
- **Escalabilidad geoespacial**: consultas eficientes de proximidad y clustering de reportes mediante índices espaciales de PostGIS.

B. Restricciones

- El backend debe ejecutarse sobre Java 21 y Spring Boot 3.5.x.
- La persistencia debe usar PostgreSQL 16 con la extensión PostGIS 3.4 para el tipo de dato geometry(Point,4326).
- Las migraciones de esquema deben ser versionadas y reproducibles mediante Flyway.
- El almacenamiento de imágenes debe soportar un proveedor en la nube (Cloudinary) intercambiable por almacenamiento local mediante un puerto común.
- El frontend debe ser una SPA en React consumible desde navegadores modernos y desplegable en Vercel.

III. ESTILO ARQUITECTÓNICO GENERAL

El sistema adopta un estilo de **monolito modular** en el backend, estructurado internamente con **Arquitectura Hexagonal** (Puertos y Adaptadores, Cockburn) por cada módulo de negocio, combinado con principios de **Domain-Driven Design** (DDD) ligero: cada módulo posee sus propias capas domain, application e infrastructure. El frontend sigue una arquitectura feature-based (organización por característica de negocio) sobre React.

Cada módulo de backend respeta las tres capas siguientes:

- **Domain:** entidades de negocio puras (sin anotaciones de framework), interfaces de repositorio (ports out) y reglas de negocio.
- **Application:** casos de uso, servicios de aplicación, DTOs, mappers y puertos de entrada (ports in, expuestos como interfaces API).
- **Infrastructure:** adaptadores concretos — controladores REST, entidades JPA, repositorios Spring Data, adaptadores externos (Cloudinary, SSE, WebSocket).

IV. VISTA DE COMPONENTES

La Figura 1 presenta el diagrama de componentes del sistema completo, mostrando el frontend SPA, el núcleo hexagonal del backend con sus siete módulos, y los sistemas externos.

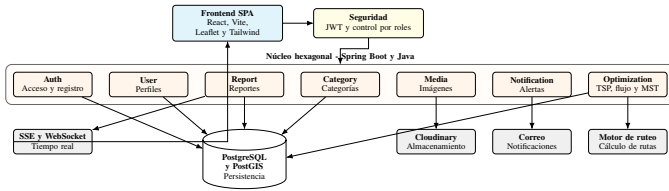


Fig. 1. Diagrama de componentes de alto nivel. El núcleo hexagonal agrupa los módulos del negocio y se comunica con los servicios externos mediante adaptadores.

A. Descripción de Módulos (Bounded Contexts)

- **auth:** gestión de cuentas, autenticación JWT y roles (CITIZEN, SUPERVISOR, ADMIN). Esquema rtc_auth.
- **user:** perfil cívico del usuario (nombre, teléfono, puntaje cívico/gamification). Esquema user.
- **report:** núcleo funcional del sistema; ciclo de vida del reporte (creación, actualización de estado, asignación a supervisor, respaldo/endorsement ciudadano, notificación en tiempo real). Esquema report.
- **category:** cataloga los tipos de incidentes y el algoritmo de optimización aplicable (ROUTING, FLOW, CONNECTIVITY, NONE). Esquema category.
- **media:** subida y gestión de evidencias fotográficas, desacoplada del proveedor de almacenamiento.
- **notification:** envío de notificaciones a usuarios ante cambios relevantes, con canal intercambiable.
- **optimization:** cálculo de rutas óptimas de atención para supervisores según el tipo de algoritmo asociado a la categoría del incidente. Esquema optimization.

V. VISTA DE DESPLIEGUE

La Figura 2 muestra el diagrama de despliegue físico/lógico del sistema en un entorno típico de producción.

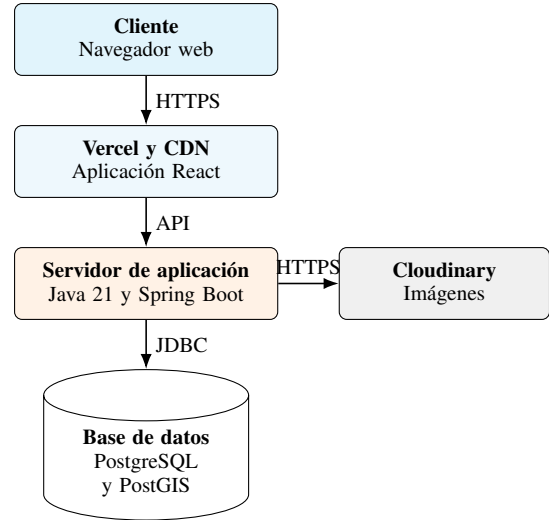


Fig. 2. Diagrama de despliegue del sistema.

VI. VISTA DE PROCESOS: FLUJO DE CREACIÓN Y NOTIFICACIÓN DE UN REPORTE

La Figura 3 ilustra el proceso extremo a extremo, desde la creación de un reporte por un ciudadano hasta la propagación en tiempo real a los tableros de administrador y otros ciudadanos suscritos.

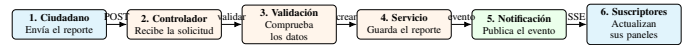


Fig. 3. Flujo de creación y notificación de un reporte.

VII. PATRONES DE DISEÑO GoF APLICADOS

Uno de los pilares de la arquitectura de ReportaTuCalle es la aplicación deliberada y manual (no derivada automáticamente de anotaciones de framework) de patrones de diseño clásicos del catálogo Gang of Four. La Tabla I resume los patrones identificados, su ubicación en el código y el problema arquitectónico que resuelven. Estos patrones se resaltan también dentro del diagrama de componentes (Fig. 1) y del modelo de clases (Sección IX).

A. Ejemplo Representativo: Strategy en Optimización de Rutas

```
1 public interface OptimizationStrategy {
2     boolean supports(AlgorithmType type);
3     Optional<OptimizedRoute> optimize(
4         Coordinate startPoint,
5         List<Coordinate> destinations);
6 }
```

Listing 1. Interfaz OptimizationStrategy (Strategy Pattern)

TABLE I
PATRONES DE DISEÑO APLICADOS EN REPORTATUCALLE

Patrón	Categoría	Implementación principal	Finalidad
Strategy	Comportamiento	OptimizationStrategy y sus estrategias concretas.	Permite cambiar el algoritmo de optimización sin modificar el servicio principal.
State	Comportamiento	ReportStatus y los estados Pending, Assigned, InProgress, Resolved y Rejected.	Encapsula las reglas y transiciones del ciclo de vida del reporte.
Factory Method	Creacional	ReportStatusFactory y NotificationFactory.	Centraliza la creación de estados y notificaciones.
Builder	Creacional	Report.Builder.	Construye reportes de forma controlada y valida los campos obligatorios.
Prototype	Creacional	ReportTemplatePrototype.	Clona plantillas de reporte reutilizables.
Chain of Responsibility	Comportamiento	ReportValidationChain, LocationValidator y ProfanityValidator.	Ejecuta validaciones sucesivas y permite añadir nuevas reglas.
Observer	Comportamiento	SseNotificationAdapter y LiveTrackingController.	Actualiza en tiempo real a los usuarios suscritos.
Adapter	Estructural	CloudinaryStorageAdapter, LocalStorageAdapter y RoutingEngineAdapter.	Adapta servicios externos a las interfaces internas del sistema.
Singleton	Creacional	AppConfigurationManager.	Mantiene una única configuración global en memoria.
Arquitectura hexagonal	Arquitectónico	Puertos del dominio y adaptadores de infraestructura.	Separa las reglas de negocio de la base de datos, la web y los servicios externos.

El servicio RouteOptimizationService recibe una lista de estrategias inyectadas por Spring y selecciona en tiempo de ejecución la implementación cuyo método supports(type) retorna verdadero, evitando condicionales acoplados a cada algoritmo concreto.

B. Ejemplo Representativo: State en el Ciclo de Vida del Reporte

```

1 public interface ReportStatus {
2     String getName();
3     boolean canUpdate();
4     boolean isActive();
5 }

```

Listing 2. Interfaz ReportStatus (State Pattern)

Cada subclase (PendingState, AssignedState, InProgressState, ResolvedState, RejectedState) encapsula sus propias reglas de transición, y ReportStatusFactory reconstruye el estado adecuado a partir del valor persistido en la columna status.

VIII. DISEÑO DE LA BASE DE DATOS

A. Enfoque General

La base de datos es PostgreSQL 16 con la extensión PostGIS 3.4, organizada en **esquemas independientes por módulo** (rtc_auth, category, "user", report, optimization), reflejando el mismo particionamiento por bounded context del código Java. Las tablas entre módulos se referencian mediante **claves foráneas blandas** (soft references, p.ej. citizen_id, account_id) sin restricción FOREIGN KEY física entre esquemas, preservando el bajo acoplamiento entre módulos a nivel de persistencia. El versionado del esquema se gestiona con **Flyway** (migraciones V1...V6).

B. Modelo Entidad-Relación

La Figura 4 presenta el modelo entidad-relación consolidado.

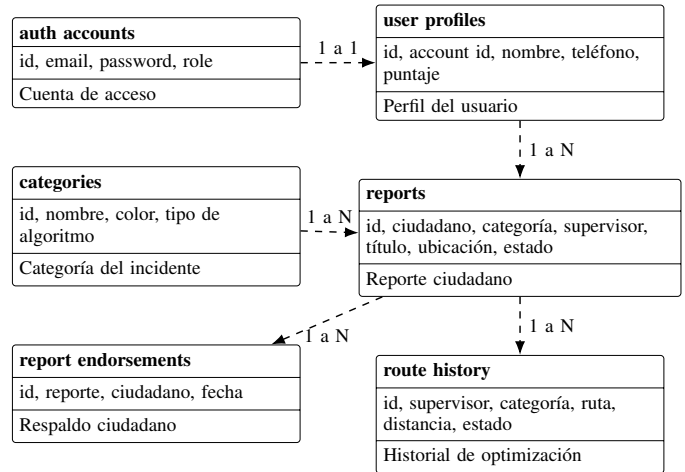


Fig. 4. Modelo entidad-relación simplificado. Las referencias entre módulos se mantienen desacopladas.

C. Diccionario de Datos

TABLE II
ESTADOS VÁLIDOS DE LOS REPORTES

Valor	Descripción
PENDING	Reporte creado y todavía sin asignar.
ASSIGNED	Reporte asignado a un supervisor.
IN PROGRESS	Incidente en atención.
RESOLVED	Incidente resuelto con evidencia.
REJECTED	Reporte descartado o inválido.

TABLE III
TIPOS DE ALGORITMO

Tipo	Uso
ROUTING	Ordena varios puntos para construir una ruta de atención.
FLOW	Analiza la capacidad de una red mediante flujo máximo.
CONNECTIVITY	Conecta puntos mediante un árbol de expansión mínima.
NONE	Muestra los reportes sin aplicar optimización.

IX. MODELO DE CLASES DEL DOMINIO

La Figura 5 presenta el modelo de clases del módulo report, el núcleo funcional del sistema, resaltando la aplicación de los patrones **Builder**, **State** y **Factory Method** descritos en la Sección VII.

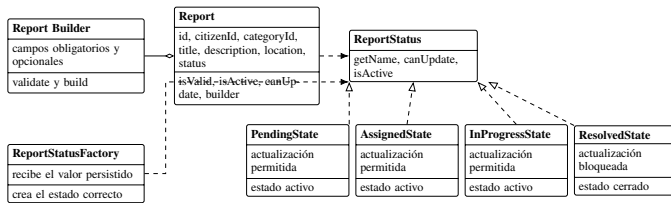


Fig. 5. Modelo de clases principal del módulo de reportes.

A. Modelo de Clases: Módulo de Optimización (Strategy)

La Figura 6 muestra el patrón Strategy aplicado a los algoritmos de optimización de rutas.

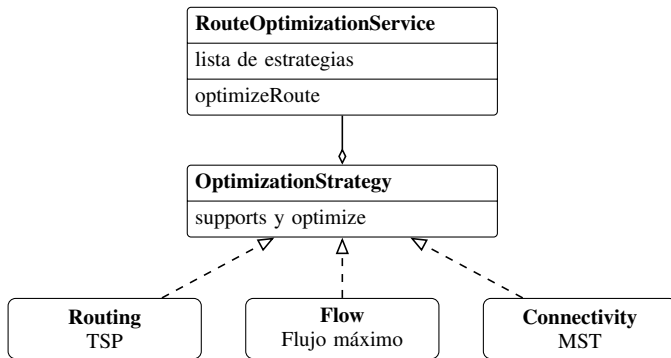


Fig. 6. Aplicación del patrón Strategy en la optimización.